

SameerBot + Nuh AI Platform

Complete Build Documentation

Project	WhatsApp AI Bot + AI Gateway
AI Name	Nuh 1.0
Created By	Sameer Shaik
Company	N.S (Nadeem Shaik)
Stack	Node.js + Spring Boot + Ollama + Docker
Date	May 2026

Table of Contents

1. Project Overview & Architecture
2. Prerequisites
3. Project Structure Setup
4. WhatsApp Bot (Baileys + Node.js)
5. Spring Boot AI Gateway
6. Ollama + Nuh Custom Model
7. Docker Setup (One Command Start)
8. Tavily Live Internet Search
9. Human Mode Feature
10. Open WebUI Integration
11. Commands Reference
12. Troubleshooting
13. Final Architecture

1. Project Overview & Architecture

SameerBot is a full-stack AI-powered WhatsApp bot built by Sameer Shaik. It uses Baileys (Node.js) to connect to WhatsApp, routes messages through a Spring Boot AI Gateway, which connects to a locally-running Ollama model (Nuh 1.0) and optionally searches the internet via Tavily API for live data.

Architecture Flow:

Layer	Technology	Purpose
WhatsApp	Baileys (Node.js)	Connect & receive/send messages
Bot Logic	bot.js	Commands, human mode, AI routing
AI Gateway	Spring Boot :8080	System prompt, Tavily search, Ollama call
AI Model	Ollama - Nuh 1.0	Language model (qwen2.5:1.5b base)
Web Search	Tavily API	Live internet search
Web UI	Open WebUI :3000	Browser chat interface
Container	Docker Compose	One-command start for all services

2. Prerequisites

Install and verify these tools before starting:

Node.js v20+ — For WhatsApp bot

```
node --version
```

Java 21 (OpenJDK) — For Spring Boot

```
java -version
```

Maven 3.8+ — For Spring build

```
mvn -version
```

Docker + Docker Compose — For containerization

```
docker --version
```

Ollama — For local AI model

```
ollama --version
```

Git — Optional, for version control

```
git --version
```

Install Ollama (Linux):

```
curl -fsSL https://ollama.com/install.sh | sh
```

Install Docker (Ubuntu):

```
sudo apt install docker.io docker-compose-plugin -y sudo usermod -aG docker $USER  
newgrp docker
```

3. Project Structure Setup

Create the workspace:

```
mkdir -p ~/Desktop/sameer-ai-platform/services cd ~/Desktop/sameer-ai-platform
```

Final folder structure:

```
sameer-ai-platform/ services/ ai-gateway/ # Spring Boot project src/main/java/... #  
ChatController.java src/main/resources/application.properties pom.xml Dockerfile  
whatsapp-bot/ # Node.js Baileys bot bot.js package.json auth/ # WhatsApp session  
(auto-created) Dockerfile docker-compose.yml # Start everything scripts/ start-all.sh
```

4. WhatsApp Bot (Baileys + Node.js)

Initialize and install dependencies:

```
cd ~/Desktop/sameer-ai-platform/services/whatsapp-bot npm init -y npm install
@whiskeysockets/baileys axios pino qrcode-terminal
```

Key package.json dependencies:

```
{ "dependencies": { "@whiskeysockets/baileys": "latest", "axios": "^1.16.1", "pino":
"^8.21.0", "qrcode-terminal": "^0.12.0" } }
```

bot.js key configuration (CONFIG object):

```
const CONFIG = { authDir: './auth', botName: 'SameerBot', aiName: 'Nuh', ownerName:
'Sameer', prefix: '!', springApiUrl: process.env.SPRING_API_URL ||
'http://localhost:8080/api/ai/chat', aiModel: 'Nuh', };
```

WhatsApp Bot Dockerfile:

```
FROM node:20-slim WORKDIR /app COPY package*.json ./ RUN npm install --omit=dev COPY
bot.js . VOLUME ["/app/auth"] CMD ["node", "bot.js"]
```

5. Spring Boot AI Gateway

The Spring Boot gateway is the brain between the WhatsApp bot and Ollama. It handles the system prompt, Tavily search injection, and routes AI calls.

application.properties:

```
spring.application.name=ai-gateway server.port=8080
spring.ai.ollama.base-url=http://localhost:11434 spring.ai.ollama.chat.model=Nuh
spring.ai.ollama.chat.options.temperature=0.7
spring.ai.tavily.api-key=tvly-your-key-here
```

ChatController.java (final version):

```
@RestController @RequestMapping("/api/ai") public class ChatController { private
final ChatClient chatClient; @Value("${spring.ai.tavily.api-key:}") private String
tavilyApiKey; public ChatController(ChatClient.Builder builder) { this.chatClient =
builder .defaultSystem("You are Nuh 1.0, created by Sameer Shaik") .build(); }
@GetMapping("/chat") public String chat(@RequestParam String message) { if
(!tavilyApiKey.isEmpty() && needsSearch(message)) { String result =
searchTavily(message); if (result != null) return result; } return
chatClient.prompt().user(message).call().content(); } }
```

Spring Dockerfile:

```
FROM eclipse-temurin:21-jre-jammy WORKDIR /app COPY target/*.jar app.jar EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the jar (always run before Docker build):

```
cd ~/Desktop/sameer-ai-platform/services/ai-gateway mvn clean package -DskipTests
-Dmaven.test.skip=true -q
```

6. Ollama + Nuh Custom Model

Nuh is a custom Ollama model built on top of qwen2.5:1.5b with a custom system prompt that gives it the correct identity.

Create the Modelfile:

```
cat > /tmp/Modelfile << 'EOF' FROM qwen2.5:1.5b SYSTEM "" You are Nuh, an AI
assistant. Version: Nuh 1.0 Created by: Sameer Shaik Company: N.S Keep replies short.
Plain text only. Use - for lists. "" PARAMETER temperature 0.7 PARAMETER top_p 0.9
EOF ollama create Nuh -f /tmp/Modelfile
```

Make Ollama listen on all interfaces (required for Docker):

```
sudo mkdir -p /etc/systemd/system/ollama.service.d sudo tee
/etc/systemd/system/ollama.service.d/override.conf << 'EOF' [Service]
Environment="OLLAMA_HOST=0.0.0.0" EOF sudo systemctl daemon-reload sudo systemctl
restart ollama # Verify (should show *:11434) ss -tlnp | grep 11434
```


7. Docker Setup (One Command Start)

docker-compose.yml (full file):

```
services: ai-gateway: build: context: ./services/ai-gateway container_name: ai-gateway ports: - "8080:8080" environment: SPRING_AI_OLLAMA_BASE_URL: http://host.docker.internal:11434 SPRING_AI_OLLAMA_CHAT_MODEL: Nuh SPRING_AI_TAVILY_API_KEY: tvly-your-key extra_hosts: - "host.docker.internal:host-gateway" restart: unless-stopped whatsapp-bot: build: context: ./services/whatsapp-bot container_name: whatsapp-bot volumes: - ./services/whatsapp-bot/auth:/app/auth environment: SPRING_API_URL: http://ai-gateway:8080/api/ai/chat depends_on: - ai-gateway restart: unless-stopped open-webui: image: ghcr.io/open-webui/open-webui:main container_name: open-webui ports: - "3000:8080" extra_hosts: - "host.docker.internal:host-gateway" environment: OLLAMA_BASE_URL: http://host.docker.internal:11434 volumes: - open-webui-data:/app/backend/data restart: unless-stopped volumes: open-webui-data:
```

Start / Stop commands:

```
# Start all services cd ~/Desktop/sameer-ai-platform sudo docker compose up -d # Stop all sudo docker compose down # View logs sudo docker compose logs -f # Rebuild after code changes sudo docker compose up -d --build
```

8. Tavily Live Internet Search

Tavily is an AI-optimized search API. When a user asks about news, weather, sports, or any live data, Spring calls Tavily first and returns the result directly — bypassing the local model which has no internet access.

Get API Key:

Sign up free at <https://app.tavily.com> — copy your API key (tvly-...)

Keywords that trigger search:

news, today, latest, current, weather, price, score, 2026, live, who won, stock, cricket, ipl, match, election, rupee, sensex

How it works in ChatController.java:

1. User sends message to WhatsApp 2. bot.js forwards to Spring: GET /api/ai/chat?message=... 3. Spring checks if message contains search keywords 4. If yes: calls Tavily API, extracts answer field 5. Returns Tavily answer directly (skips Ollama) 6. If no: sends to Nuh (Ollama) for normal AI response

Test search directly:

```
curl "http://localhost:8080/api/ai/chat?message=latest+news+today" curl  
"http://localhost:8080/api/ai/chat?message=ipl+score+today"
```

9. Human Mode Feature

Human mode allows users to request a direct conversation with Sameer. When activated, the AI (Nuh) goes silent and Sameer can reply manually from his phone. The user can wake Nuh back up at any time.

How it works:

User says	Bot action
hi / hello	Greet + show help + mention Sameer option
I want to chat with Sameer	Human mode ON — AI goes silent
talk to your boss	Human mode ON — AI goes silent
Hey Nuh, <question>	AI wakes up + answers question
Nuh what is Java	AI wakes up + answers
!help	Show all commands
!time	Show current date and time
!info	Show bot info (Nuh 1.0, Sameer Shaik, N.S)
!ask <question>	Force AI to answer
!ping	Check bot response speed

10. Open WebUI Integration

Open WebUI provides a browser-based chat interface connected to your local Ollama models. It is included in the docker-compose.yml and runs on port 3000.

Setup steps:

1. Open browser: <http://localhost:3000>
2. Create admin account (email + password, stored locally)
3. Select Nuh model from the model dropdown
4. Go to Admin Panel > Settings > Web Search
5. Enable Web Search, choose DuckDuckGo (free, no key needed)
6. Click the search icon in chat to use web search

11. Commands Reference

Command	Description	Example
!help	Show all available commands	!help
!ping	Check bot response speed	!ping
!info	Show bot info (Nuh 1.0, creator)	!info
!ask	Ask AI directly	!ask what is python
!time	Show current date and time	!time
hi/hello	Greeting with options menu	hi
I want to chat with Sameer	Activate human mode	-
Hey Nuh, ...	Wake AI from human mode	Hey Nuh, what is Java

12. Troubleshooting

Spring 404 error

Cause: Controller file was in wrong folder (outside src/main/java)

Fix: Move ChatController.java to correct package path

Spring 500 / Ollama refused

Cause: Ollama only listening on 127.0.0.1 — Docker can't reach it

Fix: Set OLLAMA_HOST=0.0.0.0 via systemd override, restart Ollama

docker-compose.yml is a directory

Cause: Previous cat > command created folder instead of file

Fix: rm -rf docker-compose.yml, then recreate as file

Bot syntax error (await outside async)

Cause: Time handler code was pasted outside any function

Fix: Remove lines before LOGGER section in bot.js

pom.xml parse error

Cause: Jackson dependency added incorrectly (missing version or duplicate)

Fix: Rewrite pom.xml from scratch with clean dependencies

jar not found by Docker

Cause: mvn build failed before docker compose up

Fix: Run: mvn clean package -DskipTests -Dmaven.test.skip=true

AI ignores search results

Cause: qwen2.5:1.5b too small to follow injected context

Fix: Return Tavily answer directly from Spring (bypass Ollama for searches)

13. Final Architecture

Service	Container	Port	Status
Spring AI Gateway	ai-gateway	8080	Always on
WhatsApp Bot	whatsapp-bot	-	Always on
Open WebUI	open-webui	3000	Always on
Ollama (Nuh)	Host machine	11434	systemd service

Daily usage — one command:

```
# Start everything cd ~/Desktop/sameer-ai-platform sudo docker compose up -d # Check status sudo docker compose ps # View live logs sudo docker compose logs -f whatsapp-bot
```

Message flow summary:

```
WhatsApp message -> bot.js (check greeting / human mode / command / AI) -> Spring GET /api/ai/chat?message=... -> Spring checks keywords (needs search?) -> YES: Tavily API -> return live answer -> NO: Ollama Nuh 1.0 -> return AI answer -> bot.js sends reply to WhatsApp
```